redrob | H2S

INDIA.RUNS

Build what next India runs on

Team Name : Snack Overflow

Team Leader Name : Jaimin Hadvani

Problem Statement : Intelligent Candidate Discovery & Ranking — Redrob AI Hiring Challenge

Solution Overview

- What is your proposed solution?

  CARA (Career-Aware Ranking Architecture) is a hybrid candidate ranking system that reads a job description, dynamically extracts its true requirements, and ranks 100,000 candidates against it in under 5 seconds — combining semantic embedding similarity (FAISS + MiniLM) with 45 engineered features covering career trajectory, skill evidence, location, education, and live platform behavior.

- What differentiates your approach from traditional candidate matching systems?
  - Reads career description text, not skill tags — keyword stuffers score near zero
  - Behavioral signals applied as a multiplier, not an additive bonus — unreachable candidates can't dominate the shortlist
  - Dynamic JD parsing means the system adapts to any job description with zero code changes
  - 5-check honeypot detector eliminates synthetic / impossible profiles before they're ever scored

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?

Must-have skills
28 core terms (embeddings, vector DB, retrieval, evaluation, NDCG/MRR)

Experience band
5–9 years, ideal 6–8 years

Location preference
Pune / Noida primary; Bengaluru, Hyderabad, Mumbai, Delhi NCR open

Disqualifiers
Pure-consulting careers, non-technical titles, research-only background

- Which candidate signals are most important for determining relevance?
- How does your solution evaluate candidate fit beyond keyword matching?
  - Career description text (production evidence) — weighted 25%, the highest single feature group
  - Semantic embedding similarity between full career history and JD — captures meaning, not just words
  - Platform-verified signals: skill assessments, GitHub activity, recruiter response rate, notice period
  - Anti-trap logic: candidates with AI skill tags but zero career-text evidence are explicitly penalized

Ranking Methodology

- How does your system retrieve, score, and rank candidates?

Hard Filter (eligibility) → **FAISS Retrieval (MiniLM embeddings)** → **45-Feature Scoring** → **Behavioral Multiplier**

- What models, algorithms, or heuristics are used?
  - sentence-transformers/all-MiniLM-L6-v2 — 384-dim career-text embeddings, pre-computed offline
  - FAISS IndexFlatIP — exact inner-product similarity search over 100K vectors
  - Rule-based 45-feature engineering — skill, career, location, education, behavioral signal groups
  - Min-max score normalization — guarantees 100 unique, strictly monotonic top-100 scores
- How are multiple candidate signals combined into a final ranking?

score = (30% embedding + 25% skill coverage + 15% trajectory + 10% YoE fit + 10% ML ratio + 5% location + 5% education − consulting penalty) × behavioral multiplier

Explainability & Data Validation

- How are ranking decisions explained?

  Every candidate gets a rule-based, template-assembled reasoning string built entirely from real profile fields — years of experience, exact title and company, specific matched skills, and live platform signals like notice period and response rate. No field is ever invented.

- How do you prevent hallucinations or unsupported justifications?
  - Reasoning generator has zero LLM calls — it is deterministic string assembly from structured data
  - Lower-ranked candidates (rank 80–100) explicitly state gaps: "limited core skills" / "[Confidence: Low]"
  - Confidence tier is computed from feature completeness, not generated free-text

- How does your solution handle inconsistent, low-quality, or suspicious profiles?
  - Skill duration exceeds total years of experience
  - 8+ expert-proficiency skills with zero endorsements
  - Proficiency contradiction (e.g. Expert PyTorch, Beginner Python)
  - Sum of career-history months impossibly exceeds stated YoE
  - Non-technical title combined with 4+ expert AI/LLM skill tags

  → **1,923 honeypots detected · 0 in final top-100**

End-to-End Workflow

What is the complete workflow from JD input to ranked candidate output?

1. JD Ingestion	Parse job_description.docx → extract skills, YoE band, locations, disqualifiers
2. Candidate Load	Stream 100,000 candidate JSONL records into memory
3. Honeypot Screen	Run 5-check detector → flag and suppress synthetic profiles
4. Feature Engineering	Compute 45 features per candidate (skill, career, location, education, behavior)
5. Embedding + FAISS	Encode career text with MiniLM → similarity search against JD vector
6. Composite Scoring	Combine features + embedding + behavioral multiplier → min-max normalize
7. Reasoning + Output	Generate top-100 reasoning strings → write submission.csv

System Architecture

PHASE 1 — OFFLINE PRE-COMPUTATION

`candidates.jsonl (100K)`

`job_description.docx`

↓ parse_job_description()

↓ detect_honeypot() × 100K

↓ engineer_features() × 100K

↓ MiniLM encode → FAISS index

`artifacts/ (features.npz, meta.json, honeypot_flags.json, candidate_ids.json)`

PHASE 2 — ONLINE RANKING

`artifacts/ (loaded instantly)`

↓ FAISS similarity lookup

↓ compute_composite_score()

↓ apply behavioral multiplier

↓ min-max normalize (unique scores)

↓ generate_reasoning() × 100

`submission.csv (100 rows)`

Results & Performance

- What results or insights demonstrate ranking quality?

Rank	Candidate	Title	Company	Score
1	CAND_0077337	Staff ML Engineer	Paytm	1.0000
2	CAND_0055905	Sr. ML Engineer	Flipkart	0.9443
3	CAND_0079387	AI Engineer	Microsoft	0.9253
4	CAND_0046525	Sr. ML Engineer	Genpact AI	0.9201
5	CAND_0007460	AI Engineer	Salesforce	0.9180

- How does your solution meet the challenge's runtime and compute constraints?

~3.5s
Online runtime (limit: 300s)

0
Honeypots in top-100

CPU-only
No GPU, no network calls

100
Unique monotonic scores guaranteed

Technologies Used

- What technologies, frameworks, and tools were used and why were they selected?

Python 3.11

Core implementation language — fast iteration, rich ML ecosystem

sentence-transformers (MiniLM-L6-v2)

Lightweight, CPU-fast embedding model — 384-dim, no GPU needed

FAISS (IndexFlatIP)

Exact inner-product similarity search — fast, deterministic, no approximation error

NumPy

Vectorized feature matrix operations across 100K candidates

python-docx

Dynamic parsing of job_description.docx — no hardcoded JD

JSON / JSONL / NPZ

Lightweight artifact storage — fast load, no database dependency

Submission Assets

GitHub Repository

github.com/your-team/CARA

Demo Video

[Insert demo video link]

Submission CSV

submission.csv — 100 ranked candidates, validated

Approach Document

Snack_Overflow_Approach.pdf

redrob | H2S

INDIA.RUNS

Build what next India runs on

THANK YOU

Team Snack Overflow · CARA — Career-Aware Ranking Architecture